



ETSI GS QKD 014 V1.1.1 (2019-02)

그룹 규격(GROUP SPECIFICATION)

양자 키 분배(QKD); REST 기반 키 전달 API 의 프로토콜 및 데이터 형식

(Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API)

면책 조항(Disclaimer)

이 문서는 양자 키 분배(QKD) ETSI 산업 규격 그룹(ISG)이 작성하고 승인하였으며, 이 ISG 에 참여한 회원들의 견해를 나타낸다. 이 문서가 반드시 ETSI 전체 회원의 견해를 나타내는 것은 아니다.

참조 번호(Reference)

DGS/QKD-014KeyDeliv

키워드(Keywords)

API, protocol, quantum cryptography, Quantum Key Distribution

ETSI

650 Route des Lucioles

F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - NAF 742 C

그라스(06) 지방청에 등록된 비영리 협회 N° 7803/88

중요 고지(Important notice)

이 문서는 <http://www.etsi.org/standards-search> 에서 내려받을 수 있다.

이 문서는 전자판 및/또는 인쇄판으로 제공될 수 있다. 이 문서의 전자판 및/또는 인쇄판의 내용은 ETSI 의 사전 서면 승인 없이 수정해서는 안 된다. 이러한 판본 및/또는 인쇄본 사이에 내용상 차이가 존재하거나 존재한다고 인식되는 경우, 우선하는 ETSI 성과물은 www.etsi.org/deliver 에서 PDF 형식으로 공개된 판본이다.

이 문서의 사용자는 이 문서가 개정되거나 상태가 변경될 수 있음에 유의해야 한다. 이 문서 및 기타 ETSI 문서의 현재 상태에 관한 정보는 <https://portal.etsi.org/TB/ETSIDeliverableStatus.aspx> 에서 확인할 수 있다.

이 문서에서 오류를 발견한 경우, 다음 서비스 중 하나로 의견을 보내기 바란다 :
<https://portal.etsi.org/People/CommitteeSupportStaff.aspx>

저작권 고지(Copyright Notification)

ETSI 의 서면 허가에 의해 승인된 경우를 제외하고, 복사 및 마이크로필름을 포함하여 전자적 또는 기계적인 어떠한 형태나 수단으로도 이 문서의 어떠한 부분도 복제하거나 이용할 수 없다. PDF 판본의 내용은 ETSI 의 서면 승인 없이 수정해서는 안 된다. 저작권 및 앞의 제한은 모든 매체에서의 복제에 적용된다.

© ETSI 2019. 모든 권리 보유.

DECT™, PLUGTESTS™, UMTS™ 및 ETSI 로고는 ETSI 가 회원의 이익을 위해 등록한 상표이다. 3GPP™와 LTE™는 ETSI 가 회원 및 3GPP 조직 파트너의 이익을 위해 등록한 상표이다. oneM2M™ 로고는 ETSI 가 회원 및 oneM2M 파트너의 이익을 위해 등록한 상표이다. GSM®과 GSM 로고는 GSM 협회가 등록하고 소유한 상표이다.

목 차

지식재산권(IPR)

머리말

조동사 용어

요약

서론

1 적용 범위

2 참고 문헌

2.1 규범적 참고 문헌

2.2 정보성 참고 문헌

3 용어, 기호 및 약어의 정의

3.1 용어

3.2 기호

3.3 약어

4 키 전달 API 규격 개요

5 프로토콜 규격

5.1 공통 규격

5.2 Get status

5.3 Get key

5.4 Get key with key IDs

6 데이터 형식 규격

6.1 Status 데이터 형식

6.2 Key request 데이터 형식

6.3 Key container 데이터 형식

6.4 Key IDs 데이터 형식

6.5 Error 데이터 형식

부속서 A (정보성) ETSI GS QKD 004 으로의 API 기능 대응

부속서 B (정보성) 다수의 SAE 에 키를 전달하는 방법의 예

부속서 C (정보성) 집필자 및 기여자

이력

지식재산권(Intellectual Property Rights)

필수 특허

규범적 성과물에 필수적이거나 필수적일 가능성이 있는 지식재산권(IPR)은 ETSI에 신고되었을 수 있다. 이러한 필수 IPR에 관한 정보가 있는 경우, 해당 정보는 ETSI 회원과 비회원 모두에게 공개되며 ETSI 사무국에서 제공하는 ETSI SR 000 314 "Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"에서 확인할 수 있다. 최신 갱신 사항은 ETSI 웹 서버(<https://ipr.etsi.org/>)에서 제공된다.

ETSI IPR 정책에 따라, ETSI는 IPR 검색을 포함한 어떠한 조사도 수행하지 않았다. ETSI SR 000 314(또는 ETSI 웹 서버의 갱신본)에 참조되지 않은 다른 IPR로서 이 문서에 필수적이거나, 필수적일 수 있거나, 향후 필수적이 될 수 있는 IPR이 존재하지 않는다는 점은 보장할 수 없다.

상표

이 문서에는 소유자가 주장하거나 등록한 상표 및/또는 상호가 포함될 수 있다. ETSI는 ETSI의 소유로 표시된 것을 제외하고는 이들에 대한 소유권을 주장하지 않으며, 상표 및/또는 상호를 사용하거나 복제할 권리를 부여하지 않는다. 이 문서에서 해당 상표를 언급하는 것이 그 상표와 관련된 제품, 서비스 또는 조직을 ETSI가 보증함을 의미하지는 않는다.

머리말(Foreword)

이 그룹 규격(GS)은 ETSI 산업 규격 그룹(ISG) 양자 키 분배(QKD)가 작성하였다.

조동사 용어(Modal verbs terminology)

이 문서에서 "shall", "shall not", "should", "should not", "may", "need not", "will", "will not", "can", "cannot"은 ETSI 작성 규칙(ETSI Drafting Rules)의 3.2 절(조항 표현을 위한 동사 형태)에 기술된 대로 해석한다.

"must"와 "must not"은 직접 인용에 사용되는 경우를 제외하고 ETSI 성과물에서 사용이 허용되지 않는다.

요약(Executive summary)

이 문서는 양자 키 분배(QKD) 네트워크가 애플리케이션에 암호 키를 공급하기 위한 통신 프로토콜과 데이터 형식을 기술한다. 이는 서로 다른 벤더의 장비 간 상호운용성을 가능하게 하는 것을 목적으로 한다. 단순하고 확장 가능하며 널리 배포되어 많은 개발자 커뮤니티에 익숙한 방식으로 REST(REpresentational State Transfer) API를 규정한다. 이 REST 기반 API는 URI의 형식, 통신 프로토콜(HTTPS), 그리고 키 자료를 포함하여 전송되는 파라미터와 응답의 JSON(JavaScript Object Notation) 데이터 형식 부호화를 규정한다.

서론(Introduction)

QKD 네트워크는 애플리케이션에 암호 키를 전달한다. QKD 네트워크, QKD 장비, 그리고 서로 다른 벤더의 애플리케이션 간 상호운용성을 보장하려면, QKD 네트워크에서 애플리케이션으로의 키 전달 API에 대한 규격이 중요하다.

또 다른 그룹 규격인 ETSI GS QKD 004 [i.1]은 애플리케이션과 QKD 키 관리 계층 사이의 객체 기반 원격 함수 호출 방식 API를 정의하며, 애플리케이션에 QoS 기능을 갖춘 키 데이터 스트림을 제공한다. 반면 이 문서는 더 단순한 키 전달 API를 정의한다. 이는 HTTPS 프로토콜과 JSON 형식으로 부호화된 데이터를 사용하여 애플리케이션에 키 ID와 함께 블록 키를 전달하는 REST 기반 API이다.

REST 기반 API는 단순하고 개발자가 이해하기 쉬우며 여러 응용 분야에서 널리 사용된다. 개발자 커뮤니티가 크고, 많은 라이브러리와 구현, 안내 문서를 커뮤니티에서 이용할 수 있다. REST 기반 API는 경량이며 노드 수와 애플리케이션 수 양쪽에서 "인터넷" 수준으로 확장된다.

이 REST 기반 키 전달 API 규격이 QKD 시장에 새로운 진입자와 개발자를 유인하고, QKD의 새로운 응용을 촉진하며, QKD를 위한 비즈니스 생태계를 발전시키는 데 기여하기를 기대한다.

1 적용 범위(Scope)

이 문서는 양자 키 분배(QKD) 네트워크가 애플리케이션에 암호 키를 공급하기 위한 통신 프로토콜과 데이터 형식을 규정한다.

이는 애플리케이션 개발자가 QKD 네트워크에 단순한 메서드 호출을 수행하여 키 자료를 전달받을 수 있게 하는 API(응용 프로그래밍 인터페이스) 형태이다. 이는 서로 다른 벤더의 장비 간 상호운용성을 가능하게 하는 것을 목적으로 한다.

QKD 네트워크는 하나의 QKD 송신기와 하나의 QKD 수신기 사이의 단일 링크로 구성될 수도 있고, 이러한 QKD 링크를 다수 포함하는 확장된 네트워크일 수도 있다. 이 API는 두 경우 모두에서 애플리케이션에 키 자료를 전달하기 위한 단일 인터페이스를 정의한다. QKD 네트워크가 원거리 노드 간에 공유되는 키 자료를 어떻게 생성하는지를 기술하는 것은 이 문서의 범위를 벗어난다.

단순하고 확장 가능하며 널리 배포되어 많은 개발자 커뮤니티에 익숙한 방식으로 REST(REpresentational State Transfer) API를 규정한다. 이 REST API는 URI의 형식, 통신 프로토콜(HTTPS), 그리고 키 자료를 포함하여 전송되는 파라미터와 응답의 JSON(JavaScript Object Notation) 데이터 형식 부호화를 규정한다.

2 참고 문헌(References)

2.1 규범적 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

예상 위치에서 공개적으로 이용할 수 없는 참조 문서는 <https://docbox.etsi.org/Reference>에서 찾을 수 있다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 이 문서의 적용에 필요하다.

- [1] IETF RFC 7230 (June 2014): "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing". (<https://www.rfc-editor.org/info/rfc7230>)
- [2] IETF RFC 7231 (June 2014): "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content". (<https://www.rfc-editor.org/info/rfc7231>)
- [3] IETF RFC 7235 (June 2014): "Hypertext Transfer Protocol (HTTP/1.1): Authentication". (<https://www.rfc-editor.org/info/rfc7235>)
- [4] IETF RFC 5246 (August 2008): "The Transport Layer Security (TLS) Protocol Version 1.2". (<https://www.rfc-editor.org/info/rfc5246>)
- [5] ETSI RFC 8446 (August 2018): "The Transport Layer Security (TLS) Protocol Version 1.3". (<https://www.rfc-editor.org/info/rfc8446>)
- [6] IETF RFC 8259 (December 2017): "The JavaScript Object Notation (JSON) Data Interchange Format". (<https://www.rfc-editor.org/info/rfc8259>)
- [7] IETF RFC 4648 (October 2006): "The Base16, Base32, and Base64 Data Encodings". (<https://www.rfc-editor.org/info/rfc4648>)

역자 주 - [5]의 "ETSI RFC 8446"은 원문 표기이며, IETF RFC 8446의 오기로 보인다.

2.2 정보성 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 이 문서의 적용에 필수적이지는 않으나, 특정 주제 영역과 관련하여 사용자에게 도움이 된다.

[i.1] ETSI GS QKD 004 (V1.1.1): "Quantum Key Distribution (QKD); Application Interface".

3 용어, 기호 및 약어의 정의

3.1 용어(Terms)

이 문서에서는 다음 용어를 적용한다.

응용 프로그래밍 인터페이스 (API, Application Programming Interface): 다른 소프트웨어 프로그램과 상호작용할 수 있도록 소프트웨어 프로그램이 구현하는 인터페이스

키(key): 전역 고유 ID 가 연관된 무작위 디지털 데이터

키 컨테이너(key container): 키 데이터를 담는 JSON 데이터 형식

주 – 6.3 절에 규정된 바와 같다.

키 관리 개체 (KME, Key Management Entity): 하나 이상의 다른 키 관리 개체와 협력하여 네트워크에서 키를 관리하는 개체

마스터 보안 애플리케이션 개체(master secure application entity): 요청에 지정된 슬레이브 보안 애플리케이션 개체가 이후에 요청할 수 있는 하나 이상의 새로운 키를, 키 관리 개체에 요청하는 보안 애플리케이션 개체

QKD 개체(QKDE, QKD Entity): QKD 프로토콜을 사용하여 적어도 하나의 다른 QKD 개체로 키를 분배하는 종단점 역할을 포함하여 키 분배 기능을 제공하는 개체

QKD 링크(QKD link): 한 쌍의 QKD 개체를 연결하는 링크

QKD 네트워크(QKD network): 둘 이상의 신뢰 노드로 구성된 네트워크

양자 키 분배(QKD, Quantum Key Distribution): 양자 정보 이론에 기초한 정보 이론적 보안을 갖는 대칭 암호 키를 생성 및 분배하는 절차 또는 방법

보안 애플리케이션 개체(SAE, Secure Application Entity): 하나 이상의 다른 보안 애플리케이션 개체와 협력하여 동작하는 하나 이상의 애플리케이션을 위해, 키 관리 개체에 하나 이상의 키를 요청하는 개체

슬레이브 보안 애플리케이션 개체(slave secure application entity): 앞서 마스터 보안 애플리케이션 개체에 전달된 하나 이상의 키 ID 를 기반으로, 키 관리 개체에 하나 이상의 키를 요청하는 보안 애플리케이션 개체

신뢰 노드(TN, Trusted Node): 보안 경계 내에 위치한 하나 이상의 키 관리 개체와 하나 이상의 QKD 개체를 포함하는 신뢰 장비를 담고 있는 노드

웹 API(web API): HTTP 또는 HTTPS 프로토콜을 사용하여 접근할 수 있는 응용 프로그래밍 인터페이스

3.2 기호(Symbols)

공란(Void).

3.3 약어(Abbreviations)

이 문서에서는 다음 약어를 적용한다.

API	Application Programming Interface
HTTP	Hyper Text Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDL	Interface Definition Language
IETF	International Engineering Task Force
JSON	JavaScript Object Notation
KME	Key Management Entity
OMG	Object Management Group
QKD	Quantum Key Distribution

QKDE	QKD Entity
QoS	Quality of Service
REST	REpresentational State Transfer
SAE	Secure Application Entity
TLS	Transport Layer Security
TN	Trusted Node
URI	Uniform Resource Identifier
URL	Unified Resource Locator
UTF-8	UCS Transformation Format 8 bits
UUID	Universally Unique Identifier

4 키 전달 API 규격 개요

이 키 전달 API 는 REST 기반 API 로, SAE 와 KME 사이의 단순한 요청-응답 방식 API 이다. SAE 는 KME 에 키 전달을 요청하고 KME 는 키를 전달한다. KME 의 API 호출은 연결 대상 KME 와 동일한 보안 사이트 내에 존재하는 SAE 가 수행하는 것을 전제로 한다. QKD 네트워크는 서로 다른 사이트의 SAE 에 공통의 공유 키를 전달할 수 있다.

광 스위치, 암호화 모듈, 보안 관리 시스템 등이 SAE 의 예이다.

키는 QKD 기술을 사용하여 KME 가 생성하고 안전하게 공유한다. KME 가 사용하는 키 관리 방법과 KME 가 QKD 네트워크에서 키를 안전하게 릴레이하는 방법은 이 문서의 범위를 벗어난다.

QKD 네트워크는 단일 QKD 링크로 구성될 수도 있고, 이러한 QKD 링크를 다수 포함하는 확장된 네트워크일 수도 있다. QKD 네트워크의 예는 (그림 1)과 같다. QKD 네트워크의 설치와 구성, 새로운 신뢰 노드나 QKD 링크의 QKD 네트워크 등록 및 제거는 QKD 네트워크 관리 사안으로서 이 문서의 범위를 벗어난다.

각 KME 는 QKD 링크를 통해 다른 KME 와 연결하기 위해 하나 또는 다수의 QKDE 를 가져야 한다. KME 는 다른 KME 로 키를 분배할 수 있어야 한다. 각 신뢰 노드에는 적어도 하나의 KME 가 있어야 한다. 하나 또는 다수의 SAE 가 신뢰 노드 내의 KME 와 연결될 수 있다. 각 신뢰 노드는 안전하게 운영되고 관리된다고 가정한다. 각 신뢰 노드는 자신의 사이트에 위치해야 한다. SAE 는 연결된 KME 와 같은 사이트에 위치해야 한다. SAE 와 KME 사이의 API 는 각 사이트의 보안 경계 내에서 사용해야 한다.

KME 는 HTTPS 프로토콜을 통해 SAE 에 키를 전달하기 위한 웹 API 서버 기능을 제공해야 한다.

각 KME 는 고유 ID(KME ID)를 가져야 한다. KME ID 는 QKD 네트워크 내에서 고유해야 한다. KME ID 의 형식과 할당 방법은 이 문서의 범위를 벗어난다.

SAE 는 키와 상태 정보를 얻기 위해 KME 에 HTTPS 요청을 수행한다.

각 SAE 는 고유 ID(SAE ID)를 가져야 한다. SAE ID 는 QKD 네트워크 내에서 고유해야 한다. SAE ID 의 형식과 할당 방법은 이 문서의 범위를 벗어난다.

SAE 와 KME 사이의 모든 통신은 HTTPS 프로토콜(TLS 버전 1.2 이상)을 사용해야 한다(IETF RFC 7230 [1], IETF RFC 7231 [2], IETF RFC 7235 [3], IETF RFC 5246 [4], IETF RFC 8446 [5]).

KME 는 각 요청을 인증하고 호출한 SAE 의 고유 SAE ID 를 식별해야 한다.

SAE 에서 KME 로 보내는 HTTPS 요청과 KME 에서 SAE 로 보내는 HTTPS 응답의 메시지 본문에 담기는 데이터는 IETF RFC 8259 [6]에 따라 JSON 형식으로 부호화해야 한다.

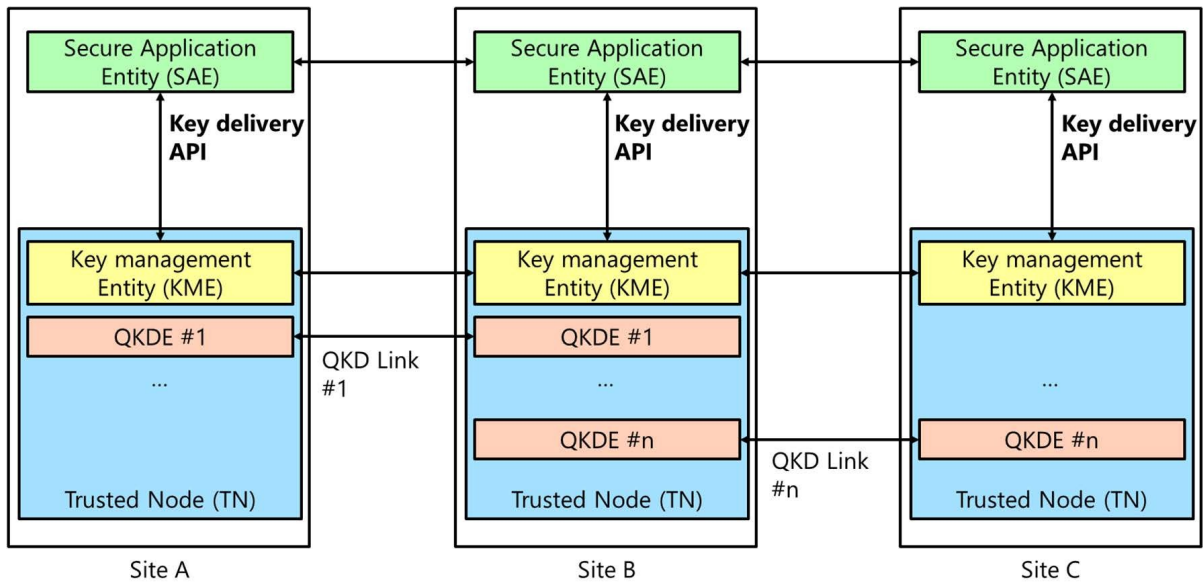
최초의 "Get key" 요청을 수행하는 SAE 를 반환되는 키에 대한 마스터 SAE 라 한다. 이어서 "Get key with key IDs" 요청을 수행하는 SAE 를 반환되는 키에 대한 슬레이브 SAE 라 한다.

애플리케이션은 동작에 필요한 만큼 SAE 간에 키 ID 를 주고받아야 하나, 그 방법은 이 문서의 범위를 벗어난다.

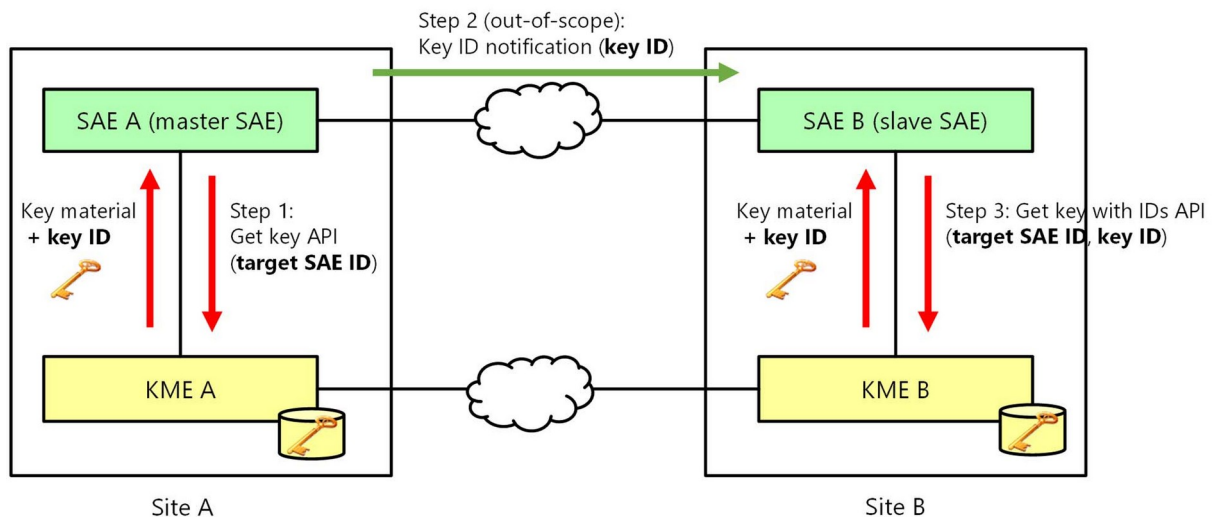
이 문서는 QKD 네트워크에서 이 API 를 사용하는 것에 대해 다음과 같은 보안 가정을 둔다.

- 각 신뢰 노드는 안전하게 운영되고 관리된다.
- 이 API 는 보안 사이트 내에서 SAE 와 KME 사이에 사용된다.

- 각 SAE 는 안전하다.
- 각 KME 는 안전하다.



(그림 1) QKD 네트워크의 예

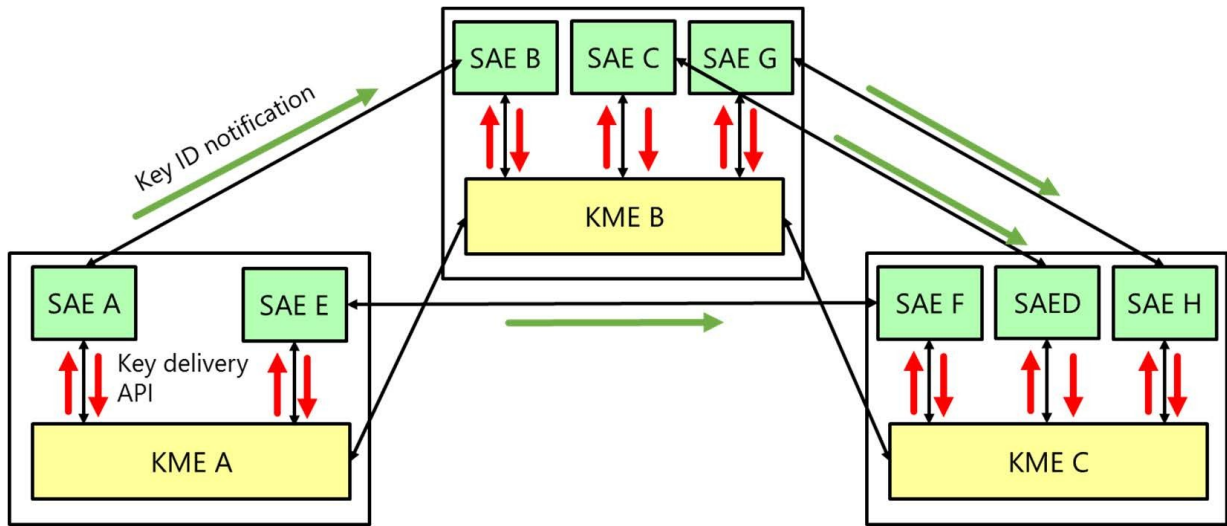


(그림 2) 키 전달 API 의 활용사례

(그림 2)는 키 전달 API 를 사용하는 방법을 보여주는 활용사례이다. KME A와 KME B는 직접 QKD 링크로 연결되거나 다수의 QKD 링크로 이루어진 QKD 네트워크로 연결된다. SAE A는 KME A에, SAE B는 KME B에 연결된다.

KME A와 KME B는 키를 교환하고 저장하며, 전달되는 각 키에는 전역 고유 ID가 부여된다. SAE A(마스터 SAE)는 다음 단계에 따라 SAE B(슬레이브 SAE)와의 보안 통신을 개시할 수 있다.

- **단계 1:** SAE A는 슬레이브 SAE인 SAE B의 ID를 지정하여 키 전달 API 메서드 "Get key"를 호출하고 KME A로부터 키를 얻는다. KME A는 KME B와 공유되는(또는 공유될) 키 자료를, 연관된 키 ID와 함께 SAE A에 전달한다.
- **단계 2:** SAE A는 SAE B에 키 ID를 알린다. 마스터 SAE와 슬레이브 SAE 간의 이 통신은 이 문서의 범위를 벗어난다.
- **단계 3:** SAE B는 마스터 SAE인 SAE A의 ID와 통지받은 키 ID 정보를 지정하여 키 전달 API 메서드 "Get key with key IDs"를 호출하고 KME B로부터 동일한 키를 얻는다. KME B는 KME A와 공유된 동일한 키 자료를 동일한 연관 키 ID와 함께 SAE B에 전달한다.



(그림 3) 각 KME 에 다수의 SAE 가 연결되는 활용사례

(그림 3)은 키 전달 API 를 사용하는 방법을 보여주는 또 다른 활용사례이다. 하나의 KME 에 다수의 SAE 가 연결된다.

5 프로토콜 규격(Protocol Specifications)

5.1 공통 규격

공통 규격은 다음과 같다.

<표 1>

Name	설명
Communication Protocol	HTTPS
Character code	UTF-8
HTTP Content-type	application/json

SAE 는 HTTPS 프로토콜(TLS 버전 1.2 이상)을 사용하여 KME 에 연결해야 한다(IETF RFC 7230 [1], IETF RFC 7231 [2], IETF RFC 7235 [3], IETF RFC 5246 [4], IETF RFC 8446 [5]). 연결 수립 시 SAE 와 KME 사이에 상호 인증을 수행해야 한다. SAE 는 KME 가 보유한 인증서의 유효성을 검증해야 하며, 그 인증서를 근거로 연결 대상 KME 의 KME ID 를 확인해야 한다. 이러한 방식으로 KME ID 가 검증되지 않으면 SAE 는 해당 KME 와 이 API 의 사용을 진행해서는 안 된다. KME 는 SAE 가 보유한 인증서의 유효성을 검증해야 하며, 그 인증서를 근거로 연결하는 SAE 의 SAE ID 를 확인해야 한다. 이러한 방식으로 KME ID 가 검증되지 않으면 KME 는 해당 SAE 의 연결을 거부해야 한다. 상호 인증 후 SAE 는 KME 의 API 메서드를 호출할 수 있다. API 메서드 목록은 다음과 같아야 한다.

역자 주 - KME 가 SAE 의 인증서를 검증하는 문장에서도 원문은 "KME ID"라고 적고 있어 그대로 옮겼으며, SAE ID 의 오키로 보인다.

<표 2>

No.	Method name	URL	Access Method
1	Get status	https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/status	GET
2	Get key	https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/enc_keys	POST (or GET)
3	Get key with key IDs	https://{KME_hostname}/api/v1/keys/{master_SAE_ID}/dec_keys	POST (or GET)

각 KME 와 각 SAE 에 인증서를 설치하거나 갱신하는 방법은 이 문서의 범위를 벗어난다.

5.2 Get status

"Get status" 메서드의 규격은 다음과 같아야 한다.

<표 3>

Name	설명
Overview	KME 가 호출한 SAE 에 Status 를 반환한다. Status 에는 지정된 슬레이브 SAE 를 대상으로 마스터 SAE 가 요청할 수 있는 키에 관한 정보가 담긴다.
Access method	GET
Access URL	https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/status
Parameters	{KME_hostname} — String (in URL): KME 의 호스트명 또는 IP 주소. 호스트명이나 IP 주소 뒤에 콜론으로 구분하여 포트 번호를 지정할 수 있다. {slave_SAE_ID} — String (in URL): 슬레이브 SAE 의 URL 부호화된 SAE ID
Request data model (from SAE to KME)	없음.
Response data model (from KME to SAE)	Status (6 절 참조)
Pre-condition	없음.
Post-condition	없음.

Get status 는 다음과 같은 오류 응답을 반환할 수 있다.

<표 4>

HTTP status code	Response data model	설명
400	Error	잘못된 요청 형식.
401	-	권한 없음.
503	Error	서버 측 오류.

5.3 Get key

"Get key" 메서드의 규격은 다음과 같아야 한다.

<표 5>

Name	설명
Overview	KME 가 호출한 마스터 SAE 에 Key container 데이터를 반환한다. Key container 데이터에는 하나 이상의 키가 담긴다. 호출한 마스터 SAE 는 Key container 데이터에 대한 요구사항을 지정하기 위해 Key request 데이터를 제공할 수 있다. slave_SAE_ID 파라미터로 지정된 슬레이브 SAE 는 이후 반환된 Key container 의 key_ID 식별자를 사용하여 원격 KME 에 대응되는 키를 요청할 수 있다.
Access method	POST (지정된 단순 요청에 한하여 GET, 6 절 참조)
Access URL	https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/enc_keys
Parameters	{KME_hostname} — String (in URL): KME 의 호스트명 또는 IP 주소. 호스트명이나 IP 주소 뒤에 콜론으로 구분하여 포트 번호를 지정할 수 있다. {slave_SAE_ID} — String (in URL): 슬레이브 SAE 의 URL 부호화된 SAE ID
Request data model (from SAE to KME)	Key request (POST 인 경우에만, 6 절 참조)
Response data model (from KME to SAE)	Key container (6 절 참조)
Pre-condition	없음.
Post-condition	SAE 에 제공된 요청 수만큼의 키가 KME 에 저장된 키 풀에서 제거된다.

"Get key" 메서드는 다음과 같은 오류 응답을 반환할 수 있다.

<표 6>

HTTP status code	Response data model	설명
400	Error	잘못된 요청 형식.
401	-	권한 없음.
503	Error	서버 측 오류.

5.4 Get key with key IDs

"Get key with key IDs" 메서드의 규격은 다음과 같아야 한다.

<표 7>

Name	설명
Overview	KME 가 호출한 슬레이브 SAE 에 Key container 를 반환한다. Key container 에는, 원격 마스터 SAE 가 Get key 호출에 대한 응답으로 받아 제공한 Key ID 를 기준으로, 앞서 그 원격 마스터 SAE 에 전달된 키와 대응되는 키가 담긴다. KME 는 요청된 Key ID 중 어느 하나라도 반환하게 된 각 "Get key" 메서드 호출 시 지정되었던 SAE ID 가 요청자의 SAE ID 가 아닌 경우, 401 HTTP 상태 코드로 요청을 거부해야 한다.
Access method	POST (지정된 단순 요청에 한하여 GET, 6 절 참조)
Access URL	https://{KME_hostname}/api/v1/keys/{master_SAE_ID}/dec_keys
Parameters	{KME_hostname} — String (in URL): KME 의 호스트명 또는 IP 주소. 호스트명이나 IP 주소 뒤에 콜론으로 구분하여 포트 번호를 지정할 수 있다. {master_SAE_ID} — String (in URL): 마스터 SAE 의 URL 부호화된 SAE ID
Request data model (from SAE to KME)	Key IDs (POST 인 경우에만, 6 절 참조)
Response data model (from KME to SAE)	Key container (6 절 참조)
Pre-condition	없음.
Post-condition	SAE 에 제공된, Key IDs 로 지정된 키가 KME 에 저장된 키 풀에서 제거된다.

"Get key with key IDs" 메서드는 다음과 같은 오류 응답을 반환할 수 있다.

<표 8>

HTTP status code	Response data model	설명
400	Error	잘못된 요청 형식.
401	-	권한 없음.
503	Error	서버 측 오류.

6 데이터 형식 규격(Data Format Specifications)

6.1 Status 데이터 형식

Status 데이터 형식은 API "Get status" 메서드의 응답 데이터 모델로 사용된다. Status 의 JSON 데이터 형식은 다음과 같아야 한다.

<표 9>

Items	Data type	설명
source_KME_ID	string	해당 KME 의 KME ID
target_KME_ID	string	대상 KME 의 KME ID
master_SAE_ID	string	호출한 마스터 SAE 의 SAE ID
slave_SAE_ID	string	지정된 슬레이브 SAE 의 SAE ID

Items	Data type	설명
key_size	integer	KME 가 SAE 에 전달할 수 있는 키의 기본 크기(비트)
stored_key_count	integer	KME 가 SAE 에 전달할 수 있는 저장된 키의 수
max_key_count	integer	stored_key_count 의 최댓값
max_key_per_request	integer	요청당 최대 키 수
max_key_size	integer	KME 가 SAE 에 전달할 수 있는 키의 최대 크기(비트)
min_key_size	integer	KME 가 SAE 에 전달할 수 있는 키의 최소 크기(비트)
max_SAE_ID_count	integer	KME 가 허용하는 additional_slave_SAE_IDs 의 최대 개수. KME 가 키 멀티캐스트를 지원하지 않는 경우 "0"
status_extension	object	(선택) 향후 사용을 위한

Status 데이터 형식의 예는 다음과 같다.

```
{
  "source_KME_ID": "AAAABBBBCCCCDDDD",
  "target_KME_ID": "EEEEFFFFGGGGHHHH",
  "master_SAE_ID": "IIIIJJJJKKKKLLLL",
  "slave_SAE_ID": "MMMMNNNN0000PPPP",
  "key_size": 352,
  "stored_key_count": 25000,
  "max_key_count": 100000,
  "max_key_per_request": 128,
  "max_key_size": 1024,
  "min_key_size": 64,
  "max_SAE_ID_count": 0
}
```

6.2 Key request 데이터 형식

Key request 데이터 형식은 API "Get key" 메서드의 요청 데이터 모델로 사용된다. Key request 의 JSON 데이터 형식은 다음과 같아야 한다.

<표 10>

Items	Data type	설명
number	integer	(선택) 요청하는 키의 수, 기본값은 1.
size	integer	(선택) 각 키의 크기(비트), 기본값은 Status 데이터 형식의 key_size 로 정의된다.
additional_slave_SAE_IDs	array of strings	(선택) 슬레이브 SAE 의 ID 배열. 둘 이상의 슬레이브 SAE 가 동일한 키를 공유하도록 지정하는 데 사용된다. ID 의 최대 개수는 Status 데이터 형식의 max_SAE_ID_count 로 정의된다.
extension_mandatory	array of objects	(선택) 이름/값 쌍으로 지정되는 확장 파라미터의 배열로, KME 는 이를 처리하거나 오류를 반환해야 한다. 파라미터 값은 객체를 포함하여 어떠한 유형이든 될 수 있다.
extension_optional	array of objects	(선택) 이름/값 쌍으로 지정되는 확장 파라미터의 배열로, KME 는 이를 무시할 수 있다. 파라미터 값은 객체를 포함하여 어떠한 유형이든 될 수 있다.

Key request 데이터 형식의 예는 다음과 같다.

```
{
  "number": 3,
  "size": 1024
}
```

```

{
  "number": 1,
  "size": 4096,
  "additional_slave_SAE_IDs": [
    "ABCDEFGH",
    "HIJKLMN"
  ]
}

{
  "number": 20,
  "size": 512,
  "extension_mandatory": [
    {
      "abc_route_type": "direct"
    },
    {
      "abc_transfer_method": "qkd"
    }
  ],
  "extension_optional": [
    {
      "abc_max_age": 30000
    }
  ]
}

```

Key request 데이터 형식의 모든 항목은 선택 사항이며, Key request의 JSON 데이터 형식은 비어 있을 수 있다.

Key request가 비어 있는 경우, SAE는 메시지 본문 없이 GET 접근 방식으로 요청을 보낼 수 있다.

Key request 데이터 형식에서 "number" 및/또는 "size"만 항목으로 있는 경우, SAE는 "number" 및/또는 "size"를 요청 URI 파라미터에 문자열 데이터로 지정하여 GET 접근 방식으로 요청을 보낼 수 있다.

예 1: GET 접근 방식을 사용하는 이러한 요청의 URL 예는 다음과 같다.

`https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/enc_keys?number=3&size=1024`

그 밖의 모든 경우에 SAE는 POST 접근 방식으로 요청을 보내야 한다.

KME는 키 크기가 8의 배수가 아닌 요청에 대해서도 키를 반환할 수 있다. 키 크기가 8의 배수인 경우만 지원하는 KME는, size 파라미터가 8의 배수가 아닌 Key request 데이터 요청에 대해 "size shall be a multiple of 8" 메시지와 함께 400 오류 응답을 반환해야 한다.

"additional_slave_SAE_IDs"는 다수의 SAE를 지정하는 데 사용된다. "additional_slave_SAE_IDs"에 하나 이상의 SAE ID가 나열된 경우, 동일한 키 자료는 마스터 SAE와 슬레이브 SAE 사이뿐만 아니라 "additional_slave_SAE_IDs"에 지정된 다른 슬레이브 SAE와도 공유되어야 한다. additional_slave_SAE_IDs를 어떻게 처리할지는 KME에 달려 있으며, 부속서 B에 예를 제시한다.

Key request 데이터 형식은 "extension_mandatory"와 "extension_optional"의 두 가지 확장 필드를 정의한다. 두 필드 모두 향후 사용을 위해 Key request 데이터 형식에 새로운 파라미터나 벤더 고유 파라미터를 도입하는 데 사용할 수 있다.

"extension_mandatory"는 KME가 요청에 대한 응답으로 키를 반환하기 위해 반드시 처리해야 하는 확장 파라미터 목록을 정의하는 데 사용된다. "extension_mandatory"에 제공된 확장 파라미터 중 하나 이상을 KME가 지원하지 않으면, KME는 "not all extension_mandatory parameters are supported" 메시지와 함께 400 오류 응답을

반환해야 한다. KME 가 "extension_mandatory" 내의 모든 확장 파라미터를 지원하지만 그 파라미터들이 지정하는 요구사항을 모두 충족하는 키를 전달할 수 없는 경우, KME 는 "not all extension_mandatory request options could be met" 메시지와 함께 400 오류 응답을 반환해야 한다.

"extension_optional" 은 KME 가 무시할 수 있는 확장 파라미터를 정의하는 데 사용된다. KME 는 "extension_optional"에 정의된 모든 선택 파라미터를 처리할 수도 있고, 하나 이상의 선택 확장 파라미터를 무시할 수도 있다. KME 가 "extension_optional" 내의 선택 확장 파라미터 중 하나 이상을 무시하는 경우, KME 는 원래 제공했을 응답(적절한 경우 Key container 데이터 형식을 포함한 200 OK 응답 포함)을 반환하거나, "not all extension_optional request options handled" 메시지와 함께 400 오류 응답을 반환할 수 있다.

"extension_mandatory"와 "extension_optional"은 SAE 에서 KME 로 어떠한 종류의 정보든 전달할 수 있다.

예 2: 필요한 키의 유형이나 향후 원하는 서비스 품질에 관한 정보.

"extension_mandatory"와 "extension_optional"은 유효한 JSON 으로 지정되기만 한다면 어떠한 이름과 유형의 명명된 파라미터든 전달하는 데 사용할 수 있다.

이 문서의 향후 버전에서는 "extension_mandatory"와 "extension_optional" 내 파라미터에 대해 예약된 파라미터 이름과 그 사용법을 규정할 수 있다. 확장 파라미터 이름 간 충돌 위험을 줄이기 위해, 확장 파라미터 이름이 이 문서에 추가되기 전까지는 밑줄로 끝나는 벤더 고유 접두사를 사용하는 것이 좋다. 하나 이상의 공통 확장 파라미터를 공유하기로 합의한 벤더 그룹은 밑줄로 끝나는 공통 접두사를 채택할 수 있다.

예 3: "Abcdefg" 사는 "abc_xxxxx"나 "abc_yyyy"와 같은 이름의 벤더 고유 확장 파라미터를 도입할 수 있다.

예 4: SAE 는 KME 에 특정 유형의 키를 요청할 수 있다. 일부 활용사례에서는 사용되는 경로의 유형이 중요할 수 있다. 벤더는 접두사 "abc_"를 사용하여, 단일 QKD 링크로 연결된 인접 신뢰 노드 간에 공유되는 키를 요청하는 abc_route_type "direct"와, SAE 가 이 API 로 연결하는 신뢰 노드 외에 적어도 하나의 다른 신뢰 노드를 경유하여 둘 이상의 QKD 링크로 릴레이되는 키를 요청하는 abc_route_type "indirect"를 정의할 수 있다. SAE 는 애플리케이션에 "indirect" abc_route_type 키가 적합하지 않은 경우 "extension_mandatory" 파라미터로 "direct" abc_route_type 키를 요청할 수 있다. SAE 가 "direct" abc_route_type 키를 선호하지만 "indirect" abc_route_type 키도 수용할 수 있다면, "extension_optional" 내에 "direct" abc_route_type 을 지정할 수 있다.

6.3 Key container 데이터 형식

Key container 데이터 형식은 API "Get key" 메서드와 "Get key with key IDs" 메서드의 응답 데이터 모델로 사용된다. Key container 의 JSON 데이터 형식은 다음과 같아야 한다.

<표 11>

Items	Data type	설명
keys	array of objects	키의 배열. 키의 수는 "Get key"의 "number" 파라미터로 지정된다. 지정되지 않은 경우 기본 키 수는 1 이다.
key_ID	string	키의 ID: UUID 형식(예: "550e8400-e29b-41d4-a716-446655440000").
key_ID_extension	object	(선택) 향후 사용을 위함
key	string	base64 [7]로 부호화된 키 데이터. 키 크기는 "Get key"의 "size" 파라미터로 지정된다. 지정되지 않은 경우 Status 데이터 모델의 "key_size" 값이 기본 크기로 사용된다.
key_extension	object	(선택) 향후 사용을 위함.
key_container_extension	object	(선택) 향후 사용을 위함.

Key container 데이터 형식의 예는 다음과 같다.

```
{
  "keys": [
```

```

{
  "key_ID": "bc490419-7d60-487f-adc1-4ddcc177c139",
  "key": "wHHVxRwDJs3/bXd38GHP3oe4svTuRpZS0yCC7x4Ly+s="
},
{
  "key_ID": "0a782fb5-3434-48fe-aa4d-14f41d46cf92",
  "key": "0eGMPxh1+2RpJpNCYixWHFLYRubp0KCw94FcCI7VdJA="
},
{
  "key_ID": "64a7e9a2-269c-4b2c-832c-5351f3ac5adb",
  "key": "479G10sfljpmfa5vn24tdzE5zqv5CafkGxYrLCK8384="
},
{
  "key_ID": "550e8400-e29b-41d4-a716-446655440000",
  "key": "csEMV9Kkmjg0PF90uc54+hykhg6iI5GTPHlP9PjgLVU="
}
]
}

```

"key_ID_extension", "key_extension", "key_container_extension"은 각각 키 ID, 키, 키 컨테이너에 관한 추가 정보를 반환하는 데 사용된다. 경우에 따라 이 확장 값들은 "Get key" 메서드 호출 시 제공된 Key request 데이터의 "extension_mandatory" 및/또는 "extension_optional" 내 확장 파라미터와 관련된 정보를 전달하는 데 사용될 수 있다.

6.4 Key IDs 데이터 형식

Key IDs 데이터 형식은 API "Get key with key IDs" 메서드의 요청 데이터 모델로 사용된다. Key IDs의 JSON 데이터 형식은 다음과 같아야 한다.

<표 12>

Items	Data type	설명
key_IDs	array of objects	키 ID의 배열
key_ID	string	키의 ID: UUID 형식(예: "550e8400-e29b-41d4-a716-446655440000")
key_ID_extension	object	(선택) 향후 사용을 위한
key_IDs_extension	object	(선택) 향후 사용을 위한

Key IDs 데이터 형식의 예는 다음과 같다.

```

{
  "key_IDs": [
    {
      "key_ID": "bc490419-7d60-487f-adc1-4ddcc177c139"
    },
    {
      "key_ID": "0a782fb5-3434-48fe-aa4d-14f41d46cf92"
    },
    {
      "key_ID": "64a7e9a2-269c-4b2c-832c-5351f3ac5adb"
    },
    {
      "key_ID": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```


}

키 ID 가 하나만 지정되고 확장이 지정되지 않은 경우, SAE 는 본문 메시지 없이 GET 접근 방식으로 요청할 수 있다. 그 밖의 경우 SAE 는 Get key with key IDs API 에 대해 Key IDs 데이터 형식과 함께 POST 접근 방식을 사용해야 한다.

예: Get key with key IDs API 에 대해 GET 접근 방식을 사용하는 접근 URL 의 예는 다음과 같다.

`https://{KME_hostname}/api/v1/keys/{slave_SAE_ID}/dec_keys?key_ID=bc490419-7d60-487f-adc1-4ddcc177c139`

키 ID 가 둘 이상 지정된 경우에는 POST 접근 방식을 사용해야 한다. POST 접근 방식은 Get key with key IDs 메서드에서 다수의 키 ID 나 Key ID 데이터 형식에 정의된 확장 필드를 지원하기 위해 사용된다.

KME 가 Key IDs 데이터 형식에 지정된 키 중 하나 이상을 반환할 수 없는 경우, KME 는 "one or more keys specified are not found on KME" 메시지와 함께 400 오류 응답을 반환해야 한다.

6.5 Error 데이터 형식

Error 데이터 형식은 API "Get status" 메서드, "Get key" 메서드, "Get key with key IDs" 메서드의 오류 응답 데이터 모델로 사용된다. Error 의 JSON 데이터 형식은 다음과 같아야 한다.

<표 13>

Items	Data type	설명
message	string	오류 메시지
details	array of objects	(선택) 이름/값 쌍으로 지정되는 추가 상세 오류 정보를 제공하기 위한 배열. 값은 객체를 포함하여 어떠한 유형이든 될 수 있다.

Error 데이터 형식의 예는 다음과 같다.

```
{
  "message": "key data access error"
}

{
  "message": "not all extension_mandatory parameters are supported",
  "details": [
    {
      "extension_mandatory_unsupported": "abc_route_type is not supported"
    }
  ]
}
```

부속서 A (정보성):

ETSI GS QKD 004 으로의 API 기능 대응

이 문서는 REST 기반 키 전달 API 이다. HTTPS 프로토콜과 JSON 데이터 형식으로 구성된 요청-응답 방식 API 이며, Get status, Get key, Get key with IDs 의 세 가지 메서드를 제공한다. 애플리케이션(마스터 SAE)은 Get key 를 호출하여 키 ID 와 함께 블록 키 자료를 얻는다. 이 키 ID 는 다른 KME 에 연결되어 있을 수 있는 애플리케이션(슬레이브 SAE)으로 전달된다. 애플리케이션(슬레이브 SAE)은 Get key with key IDs 를 호출하여 동일한 블록 키 자료를 획득한다.

반면 ETSI GS QKD 004 [i.1]은 OMG IDL 원격 함수 호출 기반 애플리케이션 인터페이스를 정의한다. 이는 애플리케이션에 QKD_OPEN, QKD_GET_KEY, QKD_CLOSE 의 세 가지 프리미티브를 제공한다. 애플리케이션은 QKD_OPEN API 를 호출하여 세션을 생성하고 key stream ID 를 얻는다. 그런 다음 그 key stream ID 로 QKD_GET_KEY 를 호출하여 데이터 스트림에서 키를 얻는다. key stream ID 는 상대방 애플리케이션으로 전달된다. 상대방 애플리케이션은 이 key stream ID 를 사용하여 동일한 키 스트림에서 동일한 키를 얻는다. 애플리케이션은 QKD_CLOSE API 를 호출하여 세션을 소멸시킨다.

이 문서는 ETSI GS QKD 004 [i.1]의 간편한 구현으로 볼 수 있다. 이 문서에서 정의한 API 를 ETSI GS QKD 004 [i.1]의 애플리케이션 인터페이스 모델에 다음과 같이 대응시킬 수 있다.

<표 A.1>

	ETSI GS QKD 014 (이 문서) API	설명		ETSI GS QKD 004 [i.1] Method	설명
1	Get key API	연관된 키 ID 와 함께 블록 키 자료를 얻음	1	QKD_OPEN	세션을 생성하고 key stream ID 를 얻음
			2	QKD_GET_KEY	지정된 key stream ID 에서 키 자료를 얻음
			3	QKD_CLOSE	세션을 소멸시킴
2	Key ID notification	범위 외	4	Key stream ID notification	범위 외
3	Get key with key IDs API	지정된 키 ID 에 대응하는 블록 키 자료를 얻음	5	QKD_OPEN	지정된 key stream ID 에 대응하는 세션을 열
			6	QKD_GET_KEY	지정된 key stream ID 에서 키 자료를 얻음
			7	QKD_CLOSE	세션을 소멸시킴

이 대응에서는 ETSI GS QKD 004 [i.1]의 key stream ID 가 이 문서의 키 ID 로 사용된다.

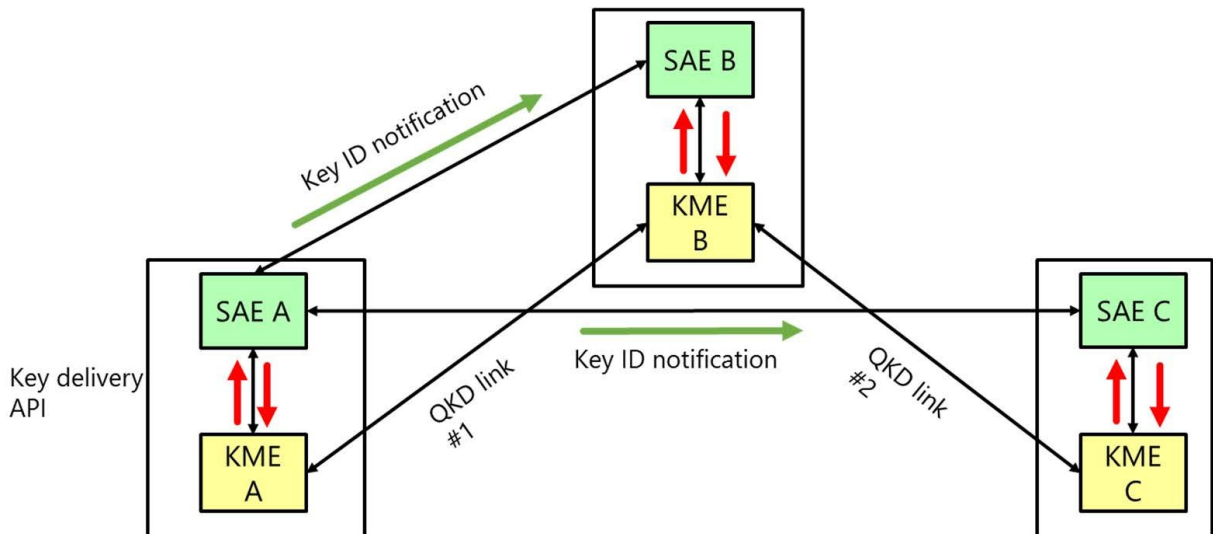
부속서 B (정보성):

다수의 SAE 에 키를 전달하는 방법의 예

이 문서는 Get key API 에서 다수의 SAE 를 지정할 수 있는 선택 파라미터를 제공한다 (6.2 절 참조). 이 선택 파라미터를 사용하면 마스터 SAE 는 요청한 키의 동일한 사본을 가져올 권한이 있는 하나 이상의 추가 슬레이브 SAE 를 지정할 수 있다. 이 기능의 구현은 KME 에 선택 사항이다. 기반 QKD 네트워크와 KME 가 다수의 SAE 에 키를 전달하는 방식은 이 문서의 범위를 벗어난다. 다만, 다음은 다수의 SAE 에 키를 전달하는 방법의 간단한 예이다.

SAE A(마스터 SAE)는 다음 단계에 따라 SAE B(슬레이브 SAE) 및 SAE C(슬레이브 SAE)와의 보안 통신을 개시할 수 있다.

- **단계 1:** SAE A는 additional_slave_SAE_IDs 선택 필드를 사용하여 슬레이브 SAE 인 SAE B 의 ID 와 SAE C 의 ID 를 지정해 키 전달 API "Get key"를 호출하고 KME A 로부터 키를 얻는다. KME A 는 연관된 키 ID 와 함께 키 자료를 생성하고, KME B 및 KME C 와 공유되는(또는 공유될) 키 자료를 SAE A 에 전달한다.
- **단계 2:** KME A 는 KME A 와 KME B 사이의 QKD 링크 #1 을 통해 공유된 QKD 키를 사용하여 생성된 키 자료를 KME B 로 안전하게 전송한다. 이어서 KME B 는 KME B 와 KME C 사이의 QKD 링크 #2 를 통해 공유된 QKD 키를 사용하여 수신한 키 자료를 KME C 로 안전하게 전송한다.
- **단계 3:** SAE A 는 SAE B 와 SAE C 에 키 ID 를 알린다.
- **단계 4:** SAE B 는 마스터 SAE 인 SAE A 의 ID 와 통지받은 키 ID 정보를 지정하여 키 전달 API "Get key with key IDs"를 호출하고 KME B 로부터 동일한 키를 얻는다. KME B 는 KME A 가 통지한 것과 일치하는 키 ID 로 요청이 오면 동일한 키 자료를 SAE B 에 전달한다.
- **단계 5:** SAE C 는 마스터 SAE 인 SAE A 의 ID 와 통지받은 키 ID 정보를 지정하여 키 전달 API "Get key with key IDs"를 호출하고 KME C 로부터 동일한 키를 얻는다. KME C 는 KME A 가 통지한 것과 일치하는 키 ID 로 요청이 오면 동일한 키 자료를 SAE C 에 전달한다.



(그림 B.1) 다수의 SAE 에 키를 전달하는 방법의 예

부속서 C (정보성):

집필자 및 기여자

다음 사람들이 이 문서에 기여하였다.

보고자(Rapporteur):

Yoshimichi Tanizawa, Toshiba

그 밖의 기여자:

Martin Ward, Toshiba

Hideaki Sato, Toshiba

Vicente Martin Ayuso, Facultad de Informatica, Universidad Politécnica de Madrid

Alejandro Aguado, Facultad de Informatica, Universidad Politécnica de Madrid

Oliver Maurhart, Austrian Institute of Technology GmbH

Alan Mink, Applied Communication Sciences - Vencore Labs, Inc.

Norbert Lutkenhaus, University of Waterloo

Momtchil Peev, Huawei Technologies Duesseldorf GmbH

Bruno Huttner, ID Quantique SA

Catherine White, British Telecommunications plc

Graham Wallace, Senetas Europe Limited

Joo Cho, ADVA Optical Networking SE

이력(History)

문서 이력

V1.1.1	2019 년 2 월	발행
--------	------------	----